

Full Stack Developer Intern -- Complete Interview Prep

Arun Kumar

Self Introduction
Part 1: JavaScript
Part 2: React (Frontend)
Part 3: Node.js & Express (Backend)
Part 4: Database -- MongoDB & Mongoose
Part 5: Authentication & Security
Part 6: HTTP, Networking & APIs
Part 7: System Design & Performance (Intern Level)
Part 8: Git & GitHub
Part 9: DSA (Expect 1-2 Live Coding Questions)
Part 10: Resume-Based Questions
Part 11: Project Deep-Dive -- AI Code Review Tool
Part 12: Project Deep-Dive -- Blogify
Part 13: Client-Server Architecture
Part 14: What is an API?
Part 15: HR / Behavioral Questions
Part 16: Questions to Ask the Interviewer
Final Tips

Self Introduction

"Hi, I'm Arun Kumar, a final-year Computer Science student at Rajkiya Engineering College, Sonbhadra. I'm currently interning at Infosys Springboard, where I work on machine learning models using Python and Scikit-learn, and apply NLP techniques for text analysis.

Alongside that, I'm passionate about full-stack development -- I build end-to-end products using the MERN stack. My flagship project is an AI Code Review Tool where I integrated Groq's LLaMA 3.3 model to give real-time code review and bug detection through a ChatGPT-like interface, with JWT authentication and persistent chat history in MongoDB. I've also built Blogify, a full-stack blogging platform with server-side rendering using EJS.

Beyond development, I'm an active competitive programmer -- I've solved 1250+ DSA problems across LeetCode, CodeChef, Codeforces and GeeksforGeeks, and I enjoy the intersection of problem-solving and building real products that people can actually use."

Part 1: JavaScript

1. Difference between var, let, and const? var is function-scoped, meaning it's accessible anywhere inside the function it's declared in, regardless of block boundaries like if or for. It also gets hoisted and initialized as undefined, so you can technically use it before its declaration line without an error. let and const are block-scoped -- they only exist within the {} they're declared in -- and they're hoisted

but placed in a "temporal dead zone," meaning accessing them before their declaration throws a `ReferenceError`. `const` additionally means the variable binding cannot be reassigned after declaration, though if it holds an object or array, the contents can still be modified -- only the reference itself is locked.

2. What is a closure? A closure happens when an inner function retains access to variables from its outer (enclosing) function's scope, even after the outer function has already finished running. This works because JavaScript functions carry a reference to their lexical scope, not a copy of the values. A classic example is a counter: an outer function declares a variable and returns an inner function that increments it -- every call to the returned function updates that same private variable. This is the mechanism behind private state in JavaScript, and is essentially how React's `useState` maintains state across renders internally.

3. Explain the event loop and how async code works. JavaScript runs on a single thread -- it can only execute one piece of code at a time. Synchronous code runs directly on the call stack. Asynchronous tasks (a `setTimeout`, a `fetch` call, a DB query) are handed off to background APIs, and JS continues executing the rest of the synchronous code without waiting. Once the background task completes, its callback goes into a queue -- either the microtask queue (Promises, `async/await`) or the macrotask queue (`setTimeout`, I/O). The event loop constantly checks: is the call stack empty? If yes, it drains the entire microtask queue first, then takes one task from the macrotask queue, and repeats. This is why a resolved Promise's `.then()` always runs before a `setTimeout(fn, 0)`.

4. Difference between `==` and `===`? `==` is loose equality -- it converts both operands to the same type before comparing, causing unexpected results (e.g. `"5" == 5` is `true`). `===` is strict equality -- compares both value and type without conversion, so `"5" === 5` is `false`. Always prefer `===` to avoid bugs from implicit coercion.

5. What are Promises, and how is `async/await` different? A Promise represents the eventual result of an async operation, existing in one of three states: pending, fulfilled, or rejected. You consume it with `.then().catch()`. `async/await` is syntactic sugar over Promises -- an async function always returns a Promise, and `await` pauses execution of that function (without blocking the whole program) until the Promise resolves, letting you write async code that reads top-to-bottom like synchronous code. Error handling is done with standard `try/catch`.

6. What is hoisting? Hoisting moves variable and function declarations to the top of their scope during compilation, before execution. Function declarations are hoisted completely and can be called before they appear in code. `var` is hoisted but holds `undefined` until its assignment line runs. `let/const` are hoisted but remain in the temporal dead zone until their declaration line executes, so early access throws an error.

7. Difference between `null` and `undefined`? `undefined` means a variable has been declared but not yet assigned a value -- JavaScript's automatic default. `null` is an intentional assignment by the developer representing "no value" deliberately.

8. What is event bubbling and event delegation? An event fired on an element "bubbles up" through all its parent elements, triggering their listeners too, unless stopped with `stopPropagation()`. Event delegation takes advantage of this by attaching one listener to a

parent container instead of every child, then using `event.target` to identify which child was actually clicked -- more memory-efficient, especially for dynamic lists.

9. Difference between `map()`, `filter()`, and `reduce()`? `map()` transforms every element and returns a new array of the same length. `filter()` returns a new array containing only elements passing a condition, so it can be shorter. `reduce()` reduces the whole array to a single accumulated value (sum, grouped object, etc.) by carrying forward an accumulator.

10. What are arrow functions and how is this different? Arrow functions don't have their own `this` -- they inherit it from the surrounding lexical scope, unlike regular functions where `this` depends on how the function is called. This is why arrow functions are preferred inside class methods and callbacks.

11. What is debouncing and throttling? Both control how often a function runs for rapidly firing events. Debouncing delays execution until a period of inactivity has passed (e.g. search-as-you-type, firing the API call only after the user stops typing). Throttling ensures a function runs at most once per fixed interval regardless of how often the event fires (e.g. scroll/resize handlers).

12. Destructuring and spread/rest operators? Destructuring unpacks values from arrays/objects into individual variables in one line. The spread operator (`...`) expands an array/object into individual elements (commonly used to copy/merge). The rest operator uses the same syntax but collects multiple values into a single array, typically in function parameters.

Part 2: React (Frontend)

1. What is the Virtual DOM and how does reconciliation work?

The real DOM is slow to update directly since changes can trigger expensive browser reflows/repaints. React keeps a lightweight in-memory copy called the Virtual DOM. On state/prop change, React builds a new Virtual DOM tree, diffs it against the previous one (reconciliation), and updates only the real DOM nodes that actually changed -- much faster than re-rendering everything.

2. Difference between `useState` and `useEffect`?

`useState` manages local component state and triggers a re-render on update. `useEffect` handles side effects (API calls, subscriptions, DOM manipulation) and runs after render -- controlled via its dependency array: `[]` runs once on mount, `[dep]` re-runs when `dep` changes, no array runs on every render.

3. Controlled vs uncontrolled components?

Controlled: input value is driven by React state (`value + onChange`), giving full control for validation. **Uncontrolled:** the DOM manages the value itself, accessed via `useRef` only when needed.

4. Difference between props and state?

Props are passed from parent to child and are read-only from the child's side. **State** is local, mutable data owned by the component itself, changing over time and triggering re-renders.

5. Why do we use key in lists, and what happens with index as key?

key gives each list item a stable identity so React can track additions/removals/reorders efficiently. Using array index as key can

cause React to misidentify items when the list reorders, leading to bugs like state sticking to the wrong item. Use a stable unique ID from the data instead.

6. What is prop drilling and how does Context API solve it? Prop drilling is passing data through several intermediate components that don't need it, just to reach a deeply nested child. Context API lets you create a shared value accessible directly by any descendant via `useContext`, without manual passing through every level -- commonly used for auth state or themes.

7. What are React hooks? Name a few. Functions that let functional components use state/lifecycle features. Common ones: `useState`, `useEffect`, `useContext`, `useRef` (persist mutable values or reference DOM elements), `useMemo` (memoize computed values), `useCallback` (memoize function references).

8. Difference between functional and class components? Class components extend `React.Component`, use `this.state` and lifecycle methods (`componentDidMount`, etc.). Functional components are plain functions returning JSX, and with hooks can manage state/side effects too -- now the modern standard due to conciseness and avoiding this binding issues.

9. What are `useMemo` and `useCallback` used for? Both prevent unnecessary recalculations. `useMemo` memoizes the result of an expensive computation, recalculating only when dependencies change. `useCallback` memoizes a function reference itself, useful when passing callbacks to children wrapped in `React.memo` to prevent unnecessary re-renders.

10. How does React handle re-rendering, and how do you optimize it? On state/prop change, React re-renders that component and its children by default, then uses Virtual DOM diffing to update only what's needed in the real DOM. Optimize using `React.memo`, `useMemo/useCallback`, splitting large components, and keeping non-essential state lower in the tree.

11. What is lazy loading / code splitting? Instead of bundling all JS into one large file, code splitting breaks it into smaller chunks, and lazy loading downloads a chunk only when needed (e.g. a settings page), using `React.lazy()` with `Suspense` for a fallback while loading.

12. What is the component lifecycle? Three main phases: Mounting (created and inserted into DOM), Updating (re-renders on changed props/state), Unmounting (removed from DOM). In functional components, `useEffect` covers all three -- the effect runs on mount/update, and its returned cleanup function runs before unmount or before the effect re-runs.

Part 3: Node.js & Express (Backend)

1. Is Node.js single-threaded? How does it handle multiple requests? Node runs JavaScript on a single main thread but uses a non-blocking, event-driven I/O model. Heavy I/O tasks (file reads, DB queries, network calls) are offloaded to the system kernel or a background thread pool (libuv), and Node picks up the result via a

callback/event once ready. This lets it handle thousands of concurrent connections without spawning a thread per request, unlike traditional multi-threaded servers.

2. What is middleware in Express? Give examples. Middleware functions sit between the incoming request and the final response, with access to `req`, `res`, and `next()`. They execute in the order they're defined, and each can either pass control forward via `next()` or end the request early. Examples: logging middleware, authentication/JWT-verification middleware, JSON body parsing (`express.json()`), and centralized error-handling middleware.

3. Difference between `req.params`, `req.query`, and `req.body`? `req.params` captures dynamic segments in the URL path (e.g. `/users/:id` -> `req.params.id`). `req.query` captures key-value pairs after `?` in the URL (e.g. `?page=2` -> `req.query.page`). `req.body` contains data sent in the request body, typically JSON, used with POST/PUT requests.

4. How do you handle errors globally in Express? By defining a special error-handling middleware at the end of the middleware chain, with four parameters (`err`, `req`, `res`, `next`). Any error passed via `next(err)` from anywhere in the app gets routed here, letting you centrally format error responses and log issues instead of repeating try/catch everywhere.

5. Difference between synchronous and asynchronous code in Node? Synchronous code executes line by line, blocking further execution until the current line finishes. Asynchronous code lets long-running operations (I/O, network calls) run in the background while the rest of the program continues, with results delivered later via callbacks, Promises, or `async/await` -- critical in Node since blocking the single thread would freeze the entire server for all users.

6. What is an event emitter in Node.js? A core Node.js pattern (the `EventEmitter` class) that allows objects to emit named events and register listener functions that respond to them. It underlies much of Node's architecture, like streams and HTTP servers, enabling a decoupled, event-driven style of programming.

7. How would you structure a Node.js project (MVC pattern)? MVC separates concerns: **Models** define data schemas and DB interaction (e.g. Mongoose schemas), **Views** represent what's sent back (JSON for an API, or templates like EJS for SSR), and **Controllers** contain the business logic that connects requests to models and sends responses. Routes typically sit separately, mapping URLs to controller functions. This keeps code organized and maintainable as the project grows.

8. What is npm vs `package.json` vs `package-lock.json`? npm (Node Package Manager) is the tool used to install and manage JavaScript packages. `package.json` lists project metadata, dependencies, and scripts. `package-lock.json` locks the exact installed versions of every dependency (including nested ones), ensuring consistent installs across different machines/environments.

9. How do environment variables (`.env`) work, and why use them? Environment variables store configuration values (API keys, DB connection strings, secrets) outside the codebase, typically in a `.env` file loaded via a package like `dotenv`. This keeps sensitive data out of version control and allows different configurations for development, staging, and production without changing code.

10. What is CORS and how did you handle it? CORS (Cross-Origin Resource Sharing) is a browser security mechanism blocking a webpage from making requests to a different domain than the one it was served from, unless the server explicitly allows it via response headers. It exists to prevent malicious sites from making requests using a logged-in user's session. In Express, this is handled using the cors middleware, configuring which origins are allowed to access the API.

Part 4: Database -- MongoDB & Mongoose

1. Difference between SQL and NoSQL databases? SQL databases are relational, use structured tables with fixed schemas, and enforce relationships via foreign keys -- good for complex, transactional data needing strong consistency. NoSQL databases like MongoDB are schema-flexible, storing JSON-like documents (BSON), and are better suited for rapidly evolving data structures and pair naturally with JavaScript objects.

2. What is indexing and why does it speed up queries? An index is a special data structure that allows the database to look up data without scanning every document/row. Without an index, MongoDB performs a full collection scan for every query. With an index on a frequently queried field (like email), lookups become dramatically faster, similar to using a book's index instead of reading every page.

3. Difference between MongoDB and Mongoose? MongoDB is the actual NoSQL database itself. Mongoose is an ODM (Object Data Modeling) library for Node.js that sits on top of MongoDB, letting you define schemas, add validation, and write queries in a more structured, developer-friendly way than the raw MongoDB driver.

4. What is a schema in Mongoose, and why is it useful in a schema-less DB? A schema defines the structure, data types, and validation rules for documents in a collection (e.g. a User must have a String email and String password). Even though MongoDB itself doesn't enforce a schema, Mongoose adds this structure at the application level, preventing malformed or inconsistent data from being saved.

5. Difference between findOne(), find(), and findById()? find() returns all documents matching a query condition, as an array. findOne() returns only the first matching document. findById() is a shortcut to find a document by its unique _id field directly.

6. What is population (populate()) in Mongoose used for? When one document references another via an ObjectId (like a blog post referencing its author's user ID), populate() automatically replaces that ID reference with the actual referenced document's data when querying -- similar to a SQL join, avoiding the need for manual separate queries.

7. What is the N+1 query problem? This happens when you fetch a list of items (1 query), then loop through them making a separate query for related data for each item (N queries) -- resulting in N+1 total queries instead of one efficient batched query. It's solved using populate() in Mongoose or batched/joined queries in SQL.

8. What is a database transaction, and does MongoDB support it? A transaction groups multiple database operations so they either all succeed together or all fail together (atomicity) -- important when multiple related writes must stay consistent (e.g. transferring money between two accounts). MongoDB supports multi-document ACID transactions starting from version 4.0, typically used within a single replica set.

9. How would you design a schema for a chat application (like your project)? A User schema (name, email, hashed password), and a Conversation/Message schema storing userId (reference), message content, sender (user or AI), and a timestamp. Indexing userId and timestamp together speeds up fetching a user's chat history in chronological order.

Part 5: Authentication & Security

1. What is authentication, and how does it work? Authentication verifies who a user is. Flow: user submits credentials -> server looks up the user record -> compares submitted password against the stored hashed password -> if it matches, issues a token (JWT) or session to prove identity for future requests, instead of asking for the password every time.

2. Difference between authentication and authorization? Authentication confirms identity ("who are you"). Authorization determines what an already-authenticated user is allowed to do ("what can you access") -- e.g. checking if a logged-in user has admin privileges.

3. What is JWT and how does it work internally? JWT (JSON Web Token) has three parts separated by dots: **header** (algorithm info), **payload** (user data like ID and expiry), and **signature** (created by signing the header+payload with a secret key). On login, the server issues this signed token. On every subsequent request, the client sends it in the Authorization header, and the server verifies the signature without needing to store any session data -- making it stateless authentication that scales well across multiple servers.

4. What is bcrypt and why do we hash passwords? Storing plain-text passwords is dangerous -- a database breach exposes everyone's password instantly. Hashing is a one-way function converting a password into irreversible gibberish; you can never get the original back, only verify a match. bcrypt specifically adds a random **salt** to each password so identical passwords produce different hashes (defeating rainbow-table attacks), and is intentionally slow (configurable cost factor) to make brute-force attacks computationally expensive.

5. Where do you store a JWT -- localStorage vs cookies -- trade-offs? localStorage is easy to access from JavaScript but vulnerable to XSS attacks (malicious scripts can read it). HTTP-only cookies can't be accessed by JavaScript, protecting against XSS, but are vulnerable to CSRF unless mitigated (e.g. with SameSite cookie attributes). Many production apps prefer HTTP-only cookies for this reason.

6. What is CSRF and XSS? How do you prevent them? **XSS (Cross-Site Scripting)**: an attacker injects malicious JavaScript into a page (e.g. via unsanitized user input) that runs in other users' browsers -- prevented by sanitizing/escaping user input and using Content Security Policy headers. **CSRF (Cross-Site Request**

Forgery): an attacker tricks a logged-in user's browser into making an unwanted request to a site they're authenticated on -- prevented using CSRF tokens and the SameSite cookie attribute.

7. What happens if a JWT is stolen? How do you mitigate that?

Since JWTs are stateless, the server can't simply "revoke" one once issued -- anyone holding it can act as that user until it expires.

Mitigations: keep expiry times short, use a **refresh token** system (a short-lived access token plus a longer-lived refresh token that can be revoked server-side), and always transmit tokens over HTTPS.

8. What is rate limiting and why did you implement it? Rate limiting restricts how many requests a user/IP can make within a time window, preventing abuse and controlling cost. In the AI Code Review Tool, each message is stored with a timestamp tied to the user's account; on a new request, the system counts messages sent in the last 4 hours (a sliding window) and rejects the request if the count exceeds 50 -- protecting against API abuse since every message triggers a paid LLM call.

9. Difference between encryption and hashing? Encryption is reversible -- data can be decrypted back to its original form using a key, used when you need to read the data again later. Hashing is one-way and irreversible, used when you only need to verify a match (like passwords), never retrieve the original value.

Part 6: HTTP, Networking & APIs

1. What happens when you enter a URL in the browser? Step by step: (1) Browser parses the URL. (2) **DNS lookup** translates the domain into an IP address -- checking browser cache, OS cache, router, then querying DNS resolvers (Root -> TLD -> Authoritative server) if not cached. (3) Browser establishes a **TCP connection** via a three-way handshake (SYN -> SYN-ACK -> ACK). (4) If HTTPS, a **TLS handshake** follows to establish encryption. (5) Browser sends an **HTTP request**. (6) Server processes it (business logic, DB queries) and sends back an **HTTP response**. (7) Browser parses the HTML, builds the DOM and CSSOM, combines them into a render tree, and paints the page, executing JavaScript along the way (where React/frontend code takes over).

2. What is HTTP? HyperText Transfer Protocol -- the rules defining how clients and servers communicate: request/response format, methods, headers. It's **stateless** -- the server doesn't remember previous requests from the same client by default, which is why we need JWTs/cookies/sessions to maintain logged-in state. Runs on port 80.

3. What is HTTPS and why does it exist? HTTP layered with TLS/SSL encryption, running on port 443. Plain HTTP sends data in readable plain text, vulnerable to man-in-the-middle interception (passwords, tokens can be stolen). HTTPS adds encryption (unreadable if intercepted), data integrity (detects tampering), and authentication (confirms you're talking to the real server via a CA-issued certificate). The TLS handshake uses asymmetric encryption briefly to securely agree on a shared symmetric key, then switches to fast symmetric encryption for the actual data transfer.

4. What is a REST API? What makes an API "RESTful"? REST (Representational State Transfer) is an architectural style for APIs using standard HTTP methods, where URLs represent resources

(/users/:id), communication is stateless (each request carries all needed info), and responses typically use JSON. "RESTful" means following these conventions consistently -- correct verb usage, resource-based naming, statelessness, and proper status codes.

5. Explain common HTTP status codes. 200 OK -- generic success. 201 Created -- new resource created (after POST). 400 Bad Request -- malformed/invalid request data. 401 Unauthorized -- not authenticated (missing/invalid token) -- "I don't know who you are." 403 Forbidden -- authenticated but not allowed -- "I know who you are, but you can't do this." 404 Not Found -- resource doesn't exist. 429 Too Many Requests -- rate limit exceeded. 500 Internal Server Error -- server-side crash/bug.

6. Difference between PUT and PATCH? PUT replaces an entire resource with the data provided -- any omitted fields may be wiped/reset. PATCH applies a partial update, modifying only the specified fields, leaving the rest untouched.

7. What is idempotency? Which HTTP methods are idempotent? An idempotent operation produces the same result no matter how many times it's repeated. GET, PUT, and DELETE are idempotent. POST is not -- calling it twice typically creates two separate resources. This matters for safe retry logic on network failures.

8. Difference between cookies, sessions, and JWT-based auth? Cookies are small pieces of data stored in the browser and sent automatically with requests to the same domain. Session-based auth stores session data server-side, giving the client just a session ID (usually via a cookie) -- stateful, server must look up the session each request. JWT-based auth is stateless -- the token itself carries the necessary user info and is verified via signature, without server-side storage.

9. What is a WebSocket, and how is it different from HTTP? HTTP follows a request-response model -- the client must initiate every exchange, and the connection typically closes after. A WebSocket establishes a persistent, two-way connection between client and server, allowing either side to send data at any time without a new request each time -- ideal for real-time features like live chat or notifications.

10. How would you version an API? Commonly by including the version in the URL path (e.g. /api/v1/users), or via a custom request header. Versioning allows you to introduce breaking changes for new clients while still supporting older client versions on the previous endpoint.

Part 7: System Design & Performance (Intern Level)

1. How would you improve a slow API? First, identify where the slowness actually is (DB query, network, payload size) via profiling/logging before optimizing blindly. Common fixes: add **database indexes** on frequently queried fields; avoid the **N+1 query problem** by batching queries; use **pagination** instead of returning huge datasets; add **caching** (e.g. Redis) for frequently requested, rarely-changing data; reduce **payload size** (only send needed fields, use gzip compression); move heavy tasks to **background job queues**

instead of making the user wait; use **connection pooling** to reuse DB connections; and use **load balancing** to distribute traffic across multiple server instances.

2. What is caching, and where would you use it (Redis)? Caching stores frequently accessed data in a fast, temporary storage layer (like Redis, which is in-memory) so repeated requests don't need to hit the database every time. Good for data that doesn't change often -- e.g. caching a product list or a user's profile info for a few minutes.

3. What is a load balancer? A component that distributes incoming traffic across multiple server instances, preventing any single server from being overwhelmed, improving both performance and reliability (if one server goes down, traffic reroutes to others).

4. How would you scale a Node.js application to handle more traffic? Run multiple instances of the app behind a load balancer (horizontal scaling), use Node's built-in `cluster` module or a process manager like PM2 to utilize multiple CPU cores, add caching layers, optimize database queries/indexes, and offload heavy/background tasks to separate worker processes or queues.

5. Difference between horizontal and vertical scaling? Vertical scaling means adding more power (CPU/RAM) to a single existing server. Horizontal scaling means adding more servers/instances and distributing load across them. Horizontal scaling is generally preferred for web applications since it avoids a single point of failure and scales further.

6. How would you design a rate limiter (like in your project)? Store a timestamp for each user action (or use a counter with a time window). On each new request, check how many actions occurred within the trailing time window (sliding window approach) -- if under the limit, allow and record it; if at/over the limit, reject with a 429 status. For higher scale, this is often implemented using Redis for fast, shared, in-memory counting across multiple server instances.

7. What is a CDN and why did you use Cloudinary? A CDN (Content Delivery Network) is a distributed network of servers that caches and delivers content (images, videos, static files) from a location geographically closer to the user, reducing load time. Cloudinary is a cloud-based media management service with built-in CDN delivery -- used in Blogify to store, optimize, resize, and fast-deliver blog images globally instead of hosting them on the app's own server.

Part 8: Git & GitHub

1. Difference between git merge and git rebase? `git merge` combines two branches by creating a new merge commit that ties both histories together, preserving the full history as it happened. `git rebase` moves/replays your branch's commits on top of another branch, creating a linear, cleaner history -- but it rewrites commit history, so it should be avoided on shared/public branches.

2. What is a pull request? A request to merge changes from one branch (often a feature branch) into another (often main), allowing team members to review code, discuss changes, and run automated checks before merging.

3. How do you resolve a merge conflict? A conflict occurs when Git can't automatically reconcile changes made to the same lines in a file across two branches. Git marks the conflicting sections in the file; you manually review both versions, decide what the final code should look like, remove the conflict markers, then stage and commit the resolved file.

4. Difference between git pull and git fetch? `git fetch` downloads new commits/branches from the remote repository but doesn't merge them into your current branch. `git pull` does both -- it fetches and then automatically merges (or rebases) the changes into your current branch.

Part 9: DSA (Expect 1-2 Live Coding Questions)

Given your 1250+ solved problems, approach these calmly -- explain your approach out loud before coding:

1. Reverse a string / array / linked list.
2. Find duplicates in an array.
3. Two Sum problem.
4. Check if a string is a palindrome.
5. Find the first non-repeating character in a string.
6. Basic recursion (factorial, Fibonacci).
7. A sliding window or two-pointer problem (conceptually similar to your rate-limiting logic).

Interview tip: Always state your approach and its time/space complexity before writing code. If stuck, talk through your thinking rather than going silent -- interviewers evaluate your problem-solving process, not just the final answer.

Part 10: Resume-Based Questions

Q: Tell me about your Infosys Springboard internship. "I'm developing machine learning models to solve real-world problems using Python and Scikit-learn. I preprocess and analyze datasets with Pandas and NumPy to improve model accuracy, and I've implemented NLP techniques for text analysis and feature extraction."

Q: Your CGPA is 7.5 -- is that a concern? "My CGPA is 7.5, and alongside academics I focused heavily on practical, industry-relevant skills -- I've built and deployed two full-stack applications and solved over 1250 DSA problems, which I believe reflect my hands-on ability well."

Q: Why did you choose the MERN stack? "MERN lets me work in one language -- JavaScript -- across the entire stack, speeding up development and removing context-switching. React gives a component-based UI, Node/Express a scalable backend, and MongoDB's flexible schema works well for fast iteration."

Q: Why Groq's LLaMA 3.3 instead of OpenAI's GPT? "Groq's LPU-based inference is extremely fast, important for a real-time, ChatGPT-like experience, and it was also more cost-effective for a project at my scale."

Part 11: Project Deep-Dive -- AI Code Review Tool

Q: Walk me through this project. "It's a full-stack AI-powered code assistant built on the MERN stack. Users paste code and get real-time code review, bug detection, and explanations from Groq's LLaMA 3.3 model, through a ChatGPT-style chat interface. I implemented persistent chat history in MongoDB so users can resume previous conversations anytime."

Q: How does authentication work here? "I used JWT -- on login the server issues a signed token, sent in the Authorization header on every protected request and verified server-side. Passwords are hashed with bcrypt before storage, so plain-text passwords are never saved."

Q: How did you implement rate limiting -- 50 messages per 4-hour window? "Each message is stored with a timestamp tied to the user's account. On a new request, I count how many messages that user sent in the last 4 hours -- a sliding window approach. If they've hit 50, the request is rejected with an appropriate error. This protects against API abuse and controls cost, since every message triggers a paid LLM call."

Q: How is chat history made persistent? "Each conversation is stored as a MongoDB document linked to the user's ID. On login or refresh, the frontend fetches past conversations from the backend and renders them, similar to ChatGPT's history."

Q: Biggest technical challenge? "Handling the AI response smoothly on the frontend -- showing the response as it streams in, rather than making the user wait for the entire reply, so it feels like real-time typing."

Q: How did you deploy it? "Frontend on Vercel, backend on Render, MongoDB Atlas as the cloud database -- separating static frontend hosting from the always-running backend server."

Q: What would you improve? "I'd add WebSockets for true real-time streaming instead of polling, and possibly a sandboxed code execution environment so users can run their code alongside getting a review."

Q: If you had to scale this to 1 million users, what would you change? "I'd add caching for repeated code patterns, move rate-limiting counters to Redis for fast shared access across multiple server instances, use a message queue for LLM requests to avoid overwhelming the API, add horizontal scaling behind a load balancer, and paginate/lazy-load chat history instead of fetching everything at once."

Part 12: Project Deep-Dive -- Blogify

Q: Explain Blogify. "It's a full-stack blogging application where users can create, manage, and publish posts. Built with Node.js, Express, and MongoDB on the backend, EJS for server-side rendered dynamic

pages, and Cloudinary for image hosting."

Q: Why EJS instead of React here? "I wanted to explore server-side rendering -- EJS renders pages on the server, which is better for SEO since search engines can directly crawl the content, and gives a faster initial page load compared to a client-heavy single-page app."

Q: Why Cloudinary? "Storing images directly on my own server isn't scalable. Cloudinary is a cloud-based media storage and CDN service that optimizes, resizes, and delivers images fast globally, so I used it for all blog post images."

Q: Role of Mongoose here? "Mongoose sits on top of MongoDB as an ODM -- I used it to define schemas for Users and Blog Posts with validation rules, making queries much cleaner than the raw MongoDB driver."

Q: Why EJS (SSR) instead of React (CSR) -- deeper trade-off?
"CSR sends a mostly empty HTML shell and builds the page via JavaScript in the browser -- faster subsequent navigation but slower initial load and weaker SEO since crawlers may not execute JS. SSR renders full HTML on the server before sending it -- faster first paint and better SEO, but more server load per request. For a content-heavy, SEO-dependent app like a blog, SSR made more sense."

Part 13: Client-Server Architecture

Client-Server architecture divides work between a **client** (requests services -- browser, mobile app) and a **server** (provides services -- processes logic, talks to the database). Flow: client sends a request over HTTP -> server processes it (validation, DB queries, business logic) -> server sends back a response -> client updates the UI.

Key characteristics: separation of concerns (frontend/backend can change independently), one server can handle many clients, stateless communication in REST APIs, and centralized data (all clients see a consistent single source of truth).

Mapped to your AI Code Review Tool: Client = React frontend (Vercel), Server = Node/Express backend (Render) that checks JWT, calls Groq's API, and talks to MongoDB Atlas. Clients never talk to the database directly -- the server acts as a controlled gatekeeper, keeping sensitive logic (API keys, DB credentials) hidden from the client, which is both a security and scalability benefit.

Part 14: What is an API?

An API (Application Programming Interface) is a set of rules allowing two software systems to communicate without either needing to know the other's internal implementation -- a contract: "send a request in this format, get a response in this format."

Restaurant analogy: You don't cook your own food (access the kitchen/database directly) -- you give your order to a waiter (the API), who takes it to the kitchen (server) and brings back your food (response).

In your project: The React frontend never talks to MongoDB or Groq directly -- it sends a request like `POST /api/chat` with a JWT token, and your Express API internally verifies the token, calls Groq, saves to MongoDB, and returns a clean JSON response -- all that complexity is hidden behind the API.

Types: REST (HTTP methods + resource URLs, stateless -- what you used), GraphQL (client specifies exact fields needed, avoiding over/under-fetching), SOAP (older, XML-based, enterprise/legacy), WebSocket (persistent two-way connection for real-time use cases).

Why APIs matter: abstraction & security (hides DB credentials/API keys), reusability (same backend can serve web, mobile, and other integrations), loose coupling (frontend/backend developed and scaled independently), and enabling integration with third-party services.

Part 15: HR / Behavioral Questions

1. **Tell me about yourself.** -- Use the intro at the top of this document.
 2. **Why do you want this internship?** -- "I want to apply my MERN stack and AI-integration experience in a real production environment, learn from experienced engineers, and contribute to solving real user problems at scale."
 3. **Why should we hire you as an intern?** -- Highlight your two live, deployed projects with real authentication/database/API integration (not just tutorials), plus your consistent problem-solving practice (1250+ DSA problems).
 4. **What are your strengths?** -- Problem-solving (1250+ DSA), fast learner (balancing ML internship + web dev), end-to-end ownership (solo-deployed apps).
 5. **What are your weaknesses?** -- Be honest, e.g.: "I sometimes spend extra time perfecting a feature, but I've been working on time-boxing tasks to stay on schedule."
 6. **Where do you see yourself in 2 years?** -- "Growing into a strong full-stack/software engineer working on scalable systems, with deeper expertise in AI integration."
 7. **Tell me about a time you were stuck on a bug -- how did you solve it?** -- Pick a real example from your projects (e.g. the AI streaming response challenge) and walk through your debugging process: isolating the issue, checking logs, searching docs, testing incrementally.
 8. **How do you manage your time between your Infosys internship, college, and personal projects?** -- Talk about prioritization and consistency, e.g. dedicating focused blocks of time and using your DSA practice as a daily habit.
 9. **Are you comfortable working in a team / taking feedback on your code?** -- "Yes -- I see code review as a way to improve, and I'm used to iterating based on feedback."
-

Part 16: Questions to Ask the Interviewer

1. "What does the day-to-day tech stack look like for interns on this team?"
2. "Will there be a mentor guiding my tasks during the internship?"

3. "Is there scope to work on AI integration within this role, or is it purely traditional web development?"
 4. "What does the code review and deployment process look like for the team?"
-

Final Tips

- Keep your intro crisp (30-45 seconds) -- don't over-explain.
- For every project decision, be ready with the "**why**" -- why this tech, why this approach. Interviewers focus heavily here.
- For coding questions: state your approach and complexity out loud before writing code.
- Stay confident -- if unsure, reason through it logically rather than immediately saying "I don't know."